
DebateIQ

A learning handbook for the developer who wrote the code with
AI
and now wants to own every line of it.

Architecture · Agentic AI · RAG · Tooling · Automation

Bugs we hit, why they happened, and the lesson behind each fix

Foreword: how to read this book

Written for the developer who built this project with an AI helper and now wants to own every line of it without that helper.

Every concept is introduced before it's used. Every design choice has a short rationale. Every bug we fixed has a paragraph explaining what broke and why. The first time a piece of jargon appears in the body text, you'll see it followed by a short italic gloss like this — **RAG** (*pulling your own material into the model prompt*) — so you never have to stop and Google.

How chapters are organised

- Each chapter starts with what the topic is in plain language.
- Then why this project needed it.
- Then how it's wired in the codebase (with file paths).
- Then a small code snippet — never a whole file, only the key idea.
- Then gotchas: things that will bite you if you change it.

Conventions used in this book

Callout — concept

A boxed idea worth committing to memory.

Tip — practical move

A way of working that will save you debugging time later.

Warning — common pitfall

Something that has actually broken on this project in the past.

Buzzword check. *An italic note like this appears when a term needs more space than an inline gloss. If you can already define the word, skim past.*

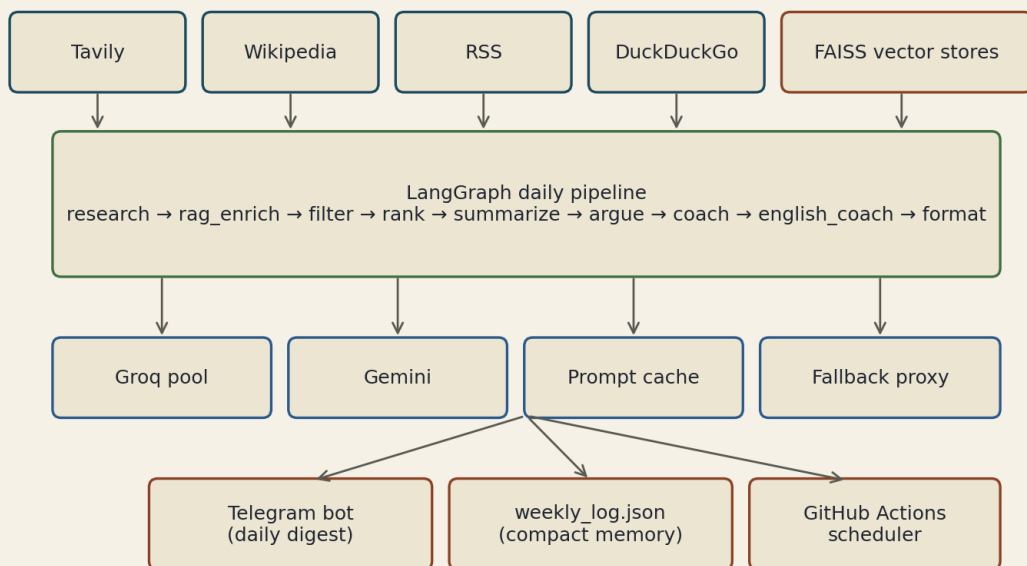
The mental model in one sentence

DebateIQ **picks one topic per day**, **researches** it through public sources and a private knowledge base, **builds a debate lesson** around it, **delivers** the lesson on Telegram, and then **reinforces** it the same night and at the end of the week.

Chapter 1

The big picture

Before any code, hold the system map in your head. Every file we talk about later sits in one of those boxes.



DebateIQ at a glance — sources at the top, a LangGraph pipeline in the middle, providers below it, outputs at the bottom.

What problem the project solves

A serious debater needs to know what's happening in the world today, what frameworks apply to it, what their best version of both sides looks like, and what specific language they should be using. Doing that by hand every morning eats two hours. DebateIQ does it on a schedule, in five minutes of compute, and delivers the result on Telegram.

Four loops that the system runs

- **Daily loop** — pulls fresh material on one topic and turns it into a structured lesson.
- **Night loop** — quizzes you on what you actually retained, or sends a compressed bedtime recap.
- **Weekend loop** — reads the whole week's memory and extracts only the concepts worth keeping.
- **Ingestion loop** — runs once when you add a PDF/site/video, builds the FAISS indexes.

The biggest architectural decision

One topic per day, not many

An earlier version of this codebase tried to cover several topics every morning. The output was unreadable and the LLM token cost was unbounded. The current design picks exactly one priority topic per day using a lightweight spaced-repetition heuristic. Every downstream module is shaped by that choice — keep this in mind.

Chapter 2

The folder structure

If a stranger walked into this codebase, the folder layout tells them what we thought were the important boundaries.

Here is ours, and why each directory exists. The dark blocks below are file paths; the comments after each one are the role that directory plays.

```
Debate Coach/
■■■■ .github/workflows/ ← Automation (CI + scheduler)
■■■■ agents/ ← One file per pipeline node
■ ■■■■ research_node.py
■ ■■■■ rag_enrich_node.py
■ ■■■■ filter_node.py
■ ■■■■ rank_node.py
■ ■■■■ summarize_node.py
■ ■■■■ argue_node.py
■ ■■■■ coach_node.py
■ ■■■■ english_coach_node.py
■ ■■■■ english_quiz_node.py
■ ■■■■ format_node.py
■ ■■■■ night_agent.py
■ ■■■■ weekend_agent.py
■ ■■■■ topic_selector.py
■■■■ core/ ← Shared plumbing
■ ■■■■ state.py ← AgentState TypedDict
■ ■■■■ llm_pool.py ← Model definitions
■ ■■■■ llm_router.py ← task_type → model key
■ ■■■■ fallback.py ← Fallback proxy
■ ■■■■ prompt_cache.py ← Disk-backed LLM cache
■ ■■■■ topic_utils.py ← topic_name(), search query expansion
■ ■■■■ network_utils.py ← Proxy env scrubber
■■■■ delivery/ ← Output adapters
■ ■■■■ telegram.py ← Live
■ ■■■■ whatsapp.py ← Shim for old imports
■■■■ rag/ ← Retrieval-augmented generation
■ ■■■■ chunking_strategy.py
■ ■■■■ embeddings.py
■ ■■■■ ingest.py
```

```

■ ■ ■ ■ retrieval_pipeline.py
■ ■ ■ ■ wpm_extractor.py
■ ■ ■ ■ sources.json
■ ■ ■ ■ memory/ ← Persistent state
■ ■ ■ ■ weekly_store.py
■ ■ ■ ■ weekly_log.json
■ ■ ■ ■ tools/ ← External research adapters
■ ■ ■ ■ tavily_tool.py
■ ■ ■ ■ wiki_tool.py
■ ■ ■ ■ rss_tool.py
■ ■ ■ ■ ddg_tool.py
■ ■ ■ ■ knowledge_base/pdfs/ ← Source material you upload
■ ■ ■ ■ faiss/ ← Built vector indexes (gitignored)
■ ■ ■ ■ tests/ ← Smoke + e2e tests
■ ■ ■ ■ graph.py ← LangGraph wiring (3 graphs)
■ ■ ■ ■ main.py ← Entry point
■ ■ ■ ■ topics.json ← Priority topics + study lens
■ ■ ■ ■ requirements.txt
■ ■ ■ ■ .env ← Secrets (gitignored)

```

Why agents/ is its own folder

Every node in the LangGraph pipeline is a pure function: it takes the shared **AgentState** dict, does its work, and returns the same dict with new fields filled in. By giving each node its own file we get three things for free: each node is easy to test in isolation, you can read the whole pipeline by reading the file names in order, and replacing a node (say, swapping the rank logic) means editing exactly one file.

Why core/ holds plumbing, not behaviour

Anything that more than one agent needs lives in **core/**. State, model pool, router, fallback, prompt cache, topic utilities, network scrubbing. Agents import from core; core never imports from agents. That one rule prevents circular imports and keeps reasoning about dependencies linear.

Why delivery/ is separate from agents/

Delivery is an I/O concern, not a thinking concern. We've already migrated delivery three times (Twilio → Meta Cloud → Telegram). Each time we only had to touch **delivery/**, never the pipeline nodes. That's what a clean boundary buys you.

Why rag/ and tools/ are separate

tools/ is for *live* data — calls go to the public internet every time. **rag/** is for *indexed* data — material you uploaded once and now retrieve by similarity. They behave very differently under failure (a live tool 429s, an index file simply isn't there) so we keep them apart.

Why memory/ is plain JSON, not a database

The whole point of memory in this project is to compound, not to scale. A 50KB JSON file holds a year of daily entries; SQLite would be overkill. Plain JSON also lets the GitHub Actions runner commit memory back to the repo trivially, which is how the night and weekend agents see history between runs.

Lesson — folders are architecture

If you can't tell what a directory is for from its name, the architecture has a leak. Resist the urge to create a misc/ or utils/ bag-of-junk folder.

Foundations you need before reading further

Concepts the rest of the book quietly assumes you already know. If any of these feel new, stop here and learn them first.

Python project basics

- **Virtual environment** (*isolated Python install per project so libraries do not collide*) — every project gets its own. We use **uv** because it resolves dependencies in one shot.
- **requirements.txt** — a pinned list of every package we need. Lower bounds ($\geq$) for flexibility, exact pins only when a version is known to break.
- **Type hints** (*function signatures that tell the editor what types are expected*) — `def f(state: dict) -> dict` is documentation the editor checks for you.
- **__init__.py** — an empty file telling Python this folder is an importable package. Every folder you import from needs one.

uv. A drop-in replacement for pip written in Rust. Installs a 50-package project in seconds instead of minutes. The command is `uv pip install ...`.

Environment variables and secrets

Anything sensitive — API keys, bot tokens, your phone number — lives in a **.env** file at the project root. **.env file** (*a plain-text file holding key=value pairs the program reads at startup*) It's listed in **.gitignore** so it never reaches the public repo.

In production those same values come from **GitHub Actions Secrets** (*encrypted values stored in your GitHub repo, available to workflows as env vars*). The code reads either source via `os.getenv()` and never cares which one

it's talking to.

Tip — read env vars lazily

Don't read `.env` at module import. Read it inside the function that actually needs it. We had a real bug where `DEV_MODE` was captured at import time, then later toggled, and the change had no effect.

Git and GitHub

- **git add / commit / push** are the three commands you'll run hundreds of times.
- Every meaningful change becomes one commit with a present-tense message: *fix: drop pre-flight ping from fallback proxy*.
- **GitHub Actions** (*GitHub built-in automation — runs scripts when you push, on a schedule, or manually*) runs our scheduler and tests. Broken CI blocks shipping.

CI / CD. Continuous Integration means tests run on every push so regressions surface immediately. Continuous Deployment means the same automation also ships the change. Our CI runs tests; our scheduler is closer to a cron job than true CD.

LangChain and LangGraph in one paragraph each

LangChain (*a Python library that wraps every LLM provider behind one common interface*) is glue. Your code calls `llm.invoke(prompt)` and stops caring whether the provider behind it is Groq, OpenAI, or Gemini. We use a tiny slice — the chat-model wrappers and the vector-store retriever interface — and write everything else ourselves.

LangGraph (*a workflow engine built on top of LangChain*) lets you declare **nodes** (functions that take state and return state) and **edges** (which node feeds which). The library handles the running order. The alternative — manually orchestrating nine functions and their failure modes — is fragile.

***LLM.** Large Language Model. A neural network trained to predict the next token of text. ChatGPT, Claude, Gemini, Llama, Qwen are all LLMs. When this book says 'the model', it means an LLM.*

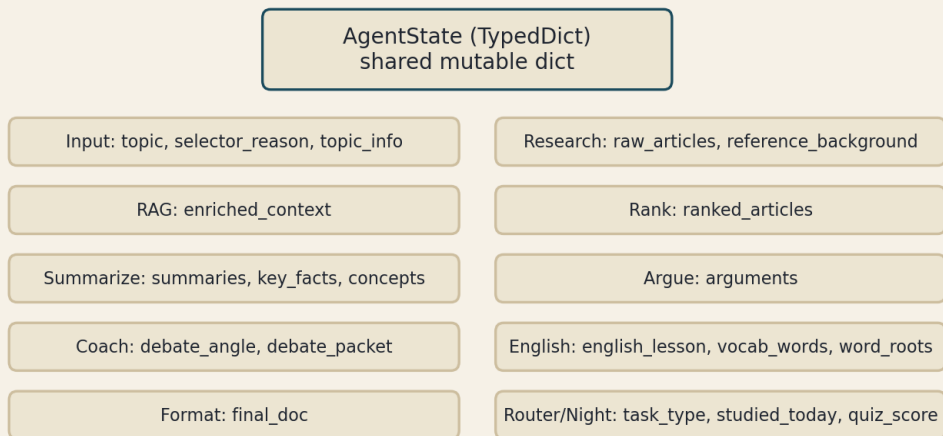
Chapter 4

Agentic AI in this project, demystified

'Agentic AI' sounds mystical. In this codebase it means: ordinary Python functions sharing one dictionary, run in a defined order.

Agentic AI. *Marketing term for systems where an LLM (or a set of them) makes decisions in a loop — read inputs, decide what to do, take an action, read the result, repeat. We use a milder version: the LLM doesn't decide the flow, but each step uses an LLM to do its work.*

Before **LangGraph** (*workflow engine that runs your nodes in order*), a Python program is a chain of function calls. After LangGraph, a Python program is still a chain of function calls — the only change is that the data passed between them lives in one shared, mutable dictionary. That dictionary is the **agent state**. Every node reads from it and writes back to it. Nothing else moves between nodes.



AgentState fields, grouped by which pipeline phase populates them.

Reading the state contract

Open `core/state.py`. The **AgentState TypedDict** (a Python dictionary type with declared keys and value types — gives you editor warnings if you mistype) is the contract between every node. If a node needs a new field — say, `english_quiz_score` — you add it here first, then every node knows it exists. If a field disappears from this file, the editor immediately tells you which nodes still reference it.

What an agent really is

In this codebase an *agent* is just a **function that takes AgentState and returns AgentState**. There's no magic. The word 'agent' is used because the function decides what to do next based on the state — for example, the night agent reads `studied_today` and picks between quiz mode and bedtime mode.

```
def example_node(state: dict) -> dict:
    # Read whatever inputs you need
    topic = state['topic']

    # Do work — LLM call, RAG lookup, computation
    result = some_work(topic)

    # Write outputs back to the same dict
    state['my_new_field'] = result
    return state
```

Why mutate, not return a new dict

Both are valid patterns. We mutate because LangGraph treats the state as a passing baton, not a copy. Mutating means every node sees every previous node's output without explicit plumbing.

Warning — never crash a node silently

If a node can't produce its output (LLM 429, retrieval empty, network down), it must still leave the field populated — usually with a heuristic fallback. A missing field will crash the next node in the chain.

The graph itself

Open *graph.py*. It declares three graphs — daily, night, weekend — by adding nodes to a *StateGraph* and drawing edges between them. For the daily graph the edges form a straight line; for the night graph it's a single node that branches at runtime based on your reply on Telegram.

```
graph = StateGraph(AgentState)
graph.add_node('research', research_node)
graph.add_node('rag_enrich', rag_enrich_node)
graph.add_edge('research', 'rag_enrich')
graph.set_entry_point('research')
compiled = graph.compile()
```

Lesson — graphs document themselves

If you can't draw your project on a whiteboard with arrows, your control flow is too tangled. Forcing the pipeline into a graph made every later bug easy to localise.

Chapter 5

The LLM layer

Two ideas tame the most unreliable part of the system: don't use your biggest model for trivial work, and have a backup plan when the primary one rejects you.

Large language models are the most expensive and most unreliable part of the system. Two patterns tame both: **role-based routing** — don't use your biggest model for trivial work — and **fallback chains** (*an ordered list of models to try when the primary one fails*) — when the primary **429s** (*HTTP status code 429 means the provider rate-limited you (too many requests or tokens)*), drop down to the next tier instead of failing the whole node.

The model pool

`core/llm_pool.py` holds the only place where a model is instantiated. Everywhere else asks for a model by *role*, never by name. That indirection means migrating from one provider to another is a one-file change.

```
LLM_POOL = {
    'fast' : llama-3.1-8b-instant, # cheap cleanup
    'balanced' : llama-3.3-70b-versatile, # summaries
    'structured': openai/gpt-oss-20b, # JSON outputs
    'reasoning' : qwen/qwen3-32b, # arguments
    'long_ctx' : gemini-2.5-flash, # long reads
    'best' : openai/gpt-oss-120b, # coach
}
```

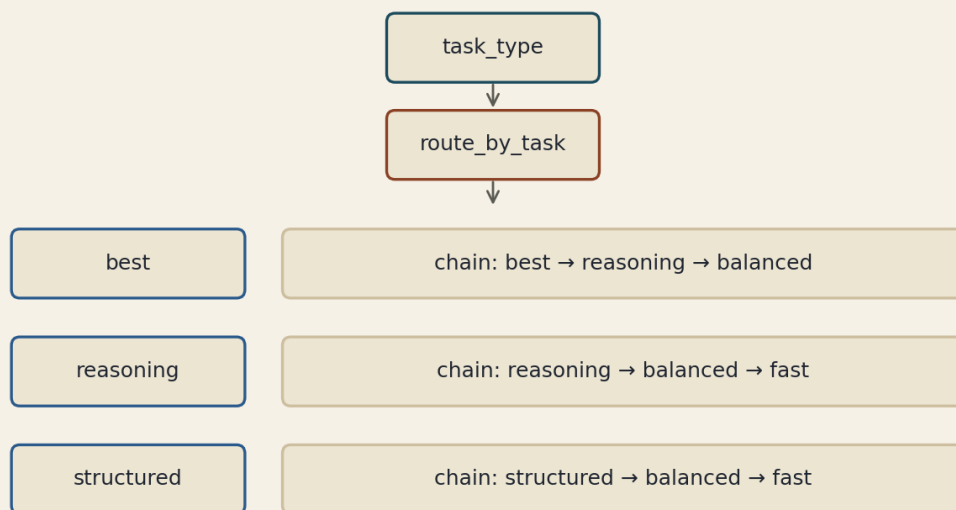
Why six lanes, not one

- **fast** handles filter + rank — yes/no decisions, no nuance needed.
- **balanced** handles summaries and bedtime — quality matters, but a 70B model is enough.
- **structured** handles JSON shapes (quiz, format). A model trained for instruction-following is more reliable here than a bigger creative model.
- **reasoning** handles argument generation. We want chain-of-thought, even visible thinking traces.
- **long_ctx** only fires when an article is huge — Gemini's million-token context shines here.
- **best** is reserved for the coach output, where one mediocre paragraph ruins the whole digest.

Routing: `task_type` → `model key`

Every node sets `state['task_type']` before calling the LLM. `core/llm_router.py` turns that string into a model key. There's a special override: if an article is longer than ~8000 chars, we force the `long_ctx` lane regardless of `task_type`.

Fallback as a transparent proxy



Each step retries only on real failure — no pre-flight ping.

get_llm_with_fallback returns a proxy. When the primary fails, the proxy walks down the chain.

core/fallback.py doesn't return the LLM directly. It returns a tiny proxy class that holds an ordered list of candidates. The proxy's *.invoke(prompt)* tries the first candidate; on any exception (rate limit, network, model decommissioned) it falls through to the next.

Bug we fixed — the pre-flight ping

The first version of the fallback proxy called *llm.invoke('ping')* before returning the model. That looked like good defensive engineering and was actively bad: it doubled token cost (every node burned an extra round trip) without actually predicting rate limits. We removed it; failures now surface on the real call.

The disk cache that saves quota

Free-tier providers cap your daily and per-minute tokens (**TPD/TPM** (*tokens per day / tokens per minute* — *provider-imposed quotas*)). The same topic comes up every ~14 days in our rotation, so the same prompts re-appear.

`core/prompt_cache.py` hashes (scope, model, prompt) into a filename in `.cache/llm/`, stores the response for 30 days, and skips the network on a hit.

Cache hit / miss. A **cache hit** means we found a stored answer for this exact prompt — return it free. A **cache miss** means we have to ask the model. The hit rate is the percentage of calls served from cache.

```
# wrap the LLM call:  
response = cached invoke(llm, prompt, scope='summarize')
```

Tip — choose your cache scope deliberately

The scope string is part of the cache key. `scope='argue'` and `scope='summarize'` never collide even if their prompts happen to match. Use one scope per node.

Research tools — the live data layer

Four ways of asking the public internet for material on today's topic.

Layered on purpose so one tool's blind spot is another's strength.

Four tools pull material from the public internet. Each one has a specific job; together they cover recency, depth, background, and redundancy.

***API.** Application Programming Interface — a contract a service exposes so other programs can talk to it. Tavily's API takes a search query and returns articles.*

Tavily — depth

Tavily is a search API tuned for LLM input. It returns full article content, not just snippets. We use it as our primary deep-search lane.

```
from langchain_tavily import TavilySearch
client = TavilySearch(max_results=5, search_depth='advanced')
```

Wikipedia — background

For any topic, Wikipedia gives you the canonical definition and a history. We never lead a digest with Wikipedia content (it reads as filler), but the rank node uses it for the BACKGROUND section.

RSS — recency

Fast-changing topics deserve a check on the last 24 hours of news. **RSS** (*Really Simple Syndication — an XML feed publishers expose listing their newest items*) feeds expose a publisher's latest items as a machine-readable list. `tools/rss_tool.py` reads a handful of curated feeds (BBC, Al Jazeera, Reuters, The Hindu, Indian Express) and filters by topic keyword.

DuckDuckGo — redundancy

No API key, no rate limits, low quality. It catches things the other three miss. Treat its results with suspicion.

Why we run them in parallel

Four serial tool calls would add ~12 seconds of latency. *research_node.py* runs them under a **ThreadPoolExecutor** (*Python helper that runs multiple functions at once on separate threads*) with a 20-second global timeout. Whoever finishes first gets included; stragglers are dropped silently.

Parallel vs concurrent. *Parallel* means truly at the same time on separate CPU cores. **Concurrent** means interleaved on one core. For network calls the distinction barely matters — your program is waiting on the network either way, so we get the speedup.

Bug we fixed — broken proxy env vars

On one machine every tool was returning empty. The diagnosis took an hour because the symptom looked like a code bug but the actual cause was `HTTP_PROXY=http://127.0.0.1:9` lingering in the shell env from a previous experiment. We now scrub local proxy env vars at startup via `core/network_utils.py`.

Why each tool normalises its output

Each adapter returns dicts with the same keys: *title*, *url*, *content*, *source*, *published*. Downstream nodes don't care which provider the article came from. If you add a fifth tool tomorrow, mirror that shape.

RAG — retrieval-augmented generation

The trick that turns a general LLM into one that knows your specific PDFs, speeches, and notes.

RAG. Retrieval-Augmented Generation. You don't fine-tune the model on your data. You search your data at query time, paste the most relevant chunks into the prompt, and let the model answer using them. Cheaper, faster, and easier to update than fine-tuning.

The mechanism is simple in three steps. (1) Take every document you care about and split it into small pieces — **chunks** (*short slices of a document — typically 200-700 characters — small enough to embed cleanly*). (2) Convert each chunk into a high-dimensional number array — an **embedding** (*a list of ~3000 numbers that represents the meaning of the text*). (3) Store the embeddings in a **vector store** (*a database optimised for finding the closest numerical neighbours to a query vector*) so you can retrieve the closest chunks to any new question.

The four vector stores

Four lanes — different content, different retrieval

knowledge_db	style_db	reasoning_db	english_db
Topic PDFs News Wikipedia	Your speeches Debate scripts Notes	Debate theory Rhetoric YouTube	Word Power Made Easy
Hybrid (BM25 40% + Vector 60%)	Similarity threshold 0.72	MMR $\lambda=0.65$, fetch_k=25	Similarity structured metadata

Each lane has different content and a different retrieval style. Not one bag of knowledge.

Hybrid retrieval — BM25 + vectors

Pure vector search is great for concepts and bad for exact terms. Searching CEDAW with embeddings might miss the chunk that has those exact letters.

BM25 (*a classic keyword-ranking algorithm from the search-engine world — predates embeddings*) does the opposite. We combine them on the knowledge store with a 40/60 weight so both win.

Hybrid retrieval. Run BM25 and vector search side by side, then blend their results. BM25 wins on exact terms (acronyms, proper nouns). Vectors win on concepts (paraphrases, synonyms). Together they cover both.

MMR — diverse chunks

MMR (*Maximum Marginal Relevance — a retrieval style that penalises redundancy in the result set*) penalises chunks that overlap with chunks already chosen. Without it, asking the reasoning store about 'fairness' might pull five chunks that all argue the same point. With MMR you get five chunks that argue five different angles. Critical for the argue node, where variety matters.

Chunking is not cosmetic

How you cut a PDF into pieces controls what retrieval can find. We use different chunkers per content type:

- **News and topic PDFs:** 600 chars, sentence-aware splitter. Facts need surrounding context.
- **Your speeches:** 400 chars. One argument point per chunk.
- **Debate theory:** 720 chars, larger overlap. Arguments span multiple sentences.
- **YouTube transcripts:** 300 chars, word-boundary. Transcripts have no punctuation structure.
- **Word Power Made Easy:** 260 chars, semicolon-aware. Each vocabulary entry stays whole.

Embeddings

We use Google's **gemini-embedding-001** (3072-dim) for every store. The earlier version used three different sentence-transformer models; we collapsed them when torch wouldn't load reliably on Windows. Single embedding model means one API surface and one cost line.

FAISS, not Chroma

Chroma (*a popular open-source vector database with a richer feature set than FAISS*) is more featureful but had build issues on Windows and the version we needed **segfaulted** (*crashed with a Segmentation Fault — a low-level memory error in native code*) under our query load. **FAISS** (*Facebook AI Similarity Search — a small, fast library for nearest-neighbour search in vector spaces*) is leaner, loads in milliseconds, and persists as two flat files per store. Migration lived in commit [a00393f](#). Lesson: pick the simpler local persistence layer unless you need the heavier semantics.

The Word Power Made Easy extractor

That PDF has 47 numbered sessions with named subsections — *TEASER PREVIEW, USING THE WORDS, ORIGINS AND RELATED WORDS, REVIEW OF ETYMOLOGY*. Generic chunking flattens all of that and embeds drill exercises alongside teaching prose. *rag/wpm_extractor.py* splits by session, then by subsection, and tags each chunk with metadata (session number, section type). The english quiz can then ask for etymology chunks specifically.

Lesson — RAG quality is 80% chunking

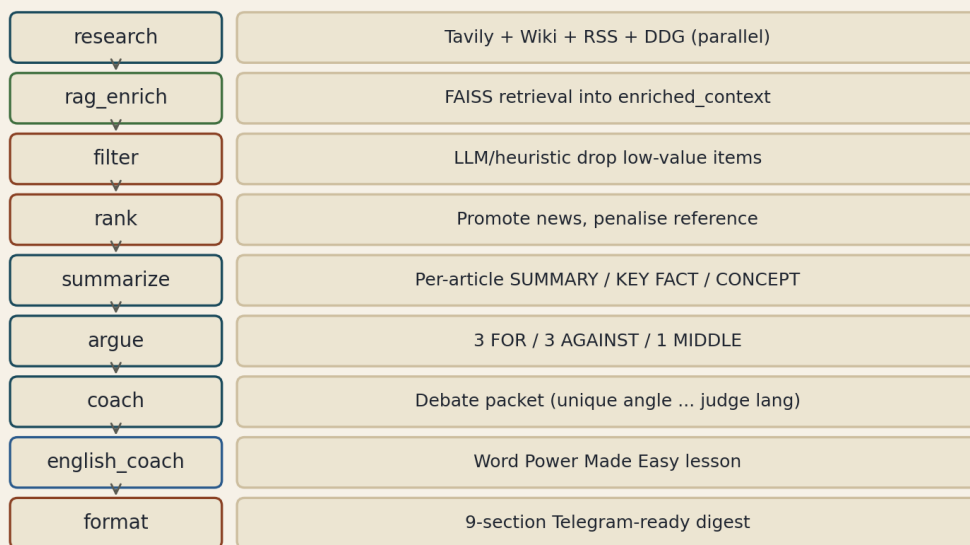
We tuned models, distance metrics, and k-values for weeks before realising the actual problem was that we were chunking a dictionary book the same way we chunk news articles. Fix the chunker first.

Chapter 8

The daily pipeline — node by node

The reference chapter. Come back here whenever you change a node or add a new one.

Each section names the file, what it reads from state, what it writes, and what its fallback does when the LLM fails.



The nine-step daily graph. Reads top-to-bottom, no branches.

research_node — gather raw material

Reads: topic, topic_info. **Writes:** raw_articles, article_length. Fires Tavily + Wikipedia + RSS + DuckDuckGo in parallel under a 20s timeout. Empty input from a single tool is normal; empty input from all four means the topic is broken or the network is down.

rag_enrich_node — inject your knowledge

Reads: topic. **Writes:** enriched_context. Runs hybrid retrieval against *knowledge_db* and MMR retrieval against *reasoning_db*. The output is one labelled string with sections for each store — the format prompt depends on those labels.

filter_node — drop noise

Reads: raw_articles. **Writes:** raw_articles (smaller). Heuristic dedup + signal-word scoring runs first. If we have enough articles, an LLM gets a JSON-only prompt to keep the strongest indices. On JSON parse failure the heuristic stands.

rank_node — pick the lead

Reads: raw_articles. **Writes:** ranked_articles. Scores each article on topic relevance, debate value, recency, source quality. News domains get a +6 boost, encyclopedia URLs get a -8 penalty. *_promote_news_first* guarantees the lead is a current-affairs piece.

summarize_node — one prompt per article

Reads: ranked_articles, enriched_context. **Writes:** summaries, key_facts, concepts. Loops the top 5 articles, sends each through the same SUMMARY/KEY FACT/CONCEPT template, parses the response. Caches each prompt by article content so re-runs don't burn tokens.

Bug we fixed — article contamination

An earlier version batched all articles into one prompt. The LLM would attribute one article's stat to another article's title. Per-article looping fixed the alignment

but cost more tokens — we balanced it by capping article content at 2500 chars and the loop at 5 articles.

argue_node — for/against/middle

Reads: topic, summaries, RAG context. **Writes:** arguments = {for, against, middle}. JSON-only prompt asking for exactly three items per side. `_normalize_arg_list` pads with heuristics if the LLM returns fewer than three.

coach_node — the debate packet

Reads: topic, arguments, RAG style chunks. **Writes:** `debate_angle`, `debate_packet`. The packet has nine named fields: unique angle, value clash, burden of proof, mechanism, opening line, claim/warrant/impact, top rebuttal, judge language, power phrases. The format node depends on those labels.

english_coach_node — Word Power lesson

Reads: topic, `english_db` retrieval. **Writes:** `english_lesson`, `vocab_words`, `word_roots`. Pulls etymology and definition chunks, asks for a JSON lesson, falls back to a curated word bank when retrieval is noisy.

format_node — assemble for delivery

Reads: everything. **Writes:** `final_doc`. Pure heuristic — no LLM call. Lays out nine sections: TOPIC FOR TODAY, PRE-KNOWLEDGE, WORD BEFORE YOU READ, TODAY'S ARTICLE / CASE, YOUR DEBATING BUILD, REBUTTAL DRILLS, WEIGHING LANGUAGE TO USE, VOCAB SESSION, THINGS TO TAKE CARE. Trims each block to a Telegram-safe size.

Drift to fix — Telegram splitter

`delivery/telegram.py` knows seven section titles. `format_node` emits nine. REBUTTAL DRILLS and WEIGHING LANGUAGE TO USE currently get glued onto the previous message. Add them to `SECTION_TITLES`.

Chapter 9

Memory — what the system remembers between runs

Without memory, every morning is the system's first morning. We keep memory deliberately small.

Without memory, the night agent would have nothing to quiz you on and the weekend agent would have nothing to distill. `memory/weekly_store.py` is a thin layer over a single JSON file.

The schema, kept deliberately small

Every daily run appends an entry keyed by today's date. The entry stores compact fields only — three bullet summaries, two key facts, three concepts, the debate angle as one string, vocab as three words. **We never store raw article bodies in memory.** Raw text would bloat the JSON to MBs and slow every read.

```
log['2026-06-05'] = [{
  'topic': 'feminism',
  'summaries': ['...', '...'],
  'key_facts': ['...'],
  'concepts': ['...'],
  'debate_angle': '...',
  'studied': False,
  'quiz_score': None,
  'timestamp': '2026-06-05T08:01:14+00:00',
}]
```

Atomic writes — no half-written files

`save_log` writes to a temp file in the same directory and then calls `os.replace` (an OS-level rename that is atomic — either fully succeeds or fully fails, never half-applied). If the runner crashes mid-write, the original log is untouched. The

temp file lives in the same dir so the rename stays atomic on every filesystem we care about.

Atomic operation. *An operation that either fully completes or has no effect at all — there's no half-done state observers can see. `os.replace` is atomic; a plain `write()` followed by another `write()` is not.*

UTC timestamps

Earlier versions used `datetime.now().isoformat()` — a **naive** (*naive datetime = no timezone attached — interpreted as local time*) local timestamp. Dev runs landed in IST, GitHub Actions in UTC. We replaced every call with `datetime.now(timezone.utc).isoformat()`. Every entry now agrees on the wall clock.

UTC. *Coordinated Universal Time — the world's primary time standard, the same everywhere. Always store and compare timestamps in UTC; convert to local time only when displaying to a human.*

Lesson — store what compounds

Memory is not a transcript. Memory is the distilled signal that next week's recap will build on. If a field doesn't help the weekend agent, it doesn't belong in the log.

Night and weekend loops — the reinforcement layer

Without these two loops the system is just a newsletter. With them it's a learning tool.

Information delivered in the morning is forgotten by evening. The night agent and weekend agent exist to make today's lesson stick.

Night agent — three-way branch

At 10:30 PM the scheduler sends you a check-in. Your reply determines the path:

- **yes** → MCQ debate quiz built from today's memory.
- **english** → vocabulary quiz from the Word Power store.
- **no** or no reply → compressed 100-word bedtime recap.

The debate quiz is a real five-question multiple-choice exam, scored by exact letter match (*1a 2b 3c 4d 5a*). The LLM builds the quiz from `today_log`; on token-bloat 413 errors the heuristic MCQ builder takes over.

Weekend agent — distill the week

Sunday morning the scheduler reads the last seven days of memory and asks the reasoning model to filter ruthlessly: keep only concepts, frameworks, statistics, and argument patterns. Anything topical, anecdotal, or shelf-life-bound gets dropped.

Spaced repetition without the framework

`agents/topic_selector.py` sorts priority topics by (seen_today_penalty, -days_since, appearances, index). Topics not studied for the longest time bubble to the top; topics already studied today sink. Conceptually it's **spaced repetition** (*showing material at increasing intervals so you review it just as you are about to forget — the principle behind Anki, Quizlet, Duolingo*) — the implementation is twenty lines.

Tip — reinforcement is the point

Without the night and weekend loops this project is just a newsletter. With them it's a learning system. Treat them as first-class features, not nice-to-haves.

Delivery — how the lesson reaches your phone

Delivery looks trivial. We migrated providers three times before landing on something that actually worked end to end.

Every migration only touched *delivery/*. The pipeline never knew the channel had changed. That's the payoff of keeping I/O out of your business logic.

Phase one — Twilio WhatsApp sandbox

Twilio is the textbook choice. We started there. Two problems killed it: trial accounts cap out fast, and the sandbox would silently downgrade WhatsApp messages to SMS when the channel prefix wasn't exactly right. The bug looked like the code; the cause was a config string.

Phase two — Meta WhatsApp Cloud API

Better economics (1000 free conversations per month), but Meta's reply detection requires a public **webhook** (*a URL on a publicly-reachable server that the provider calls when something happens — needs hosting and a stable IP*). GitHub Actions runners don't have a public IP. We considered standing up a free-tier Flask server, then asked whether the complexity was worth it.

Phase three — Telegram bot

Telegram has no conversation cap, no 24-hour session window, and a **pull-based** (*your code asks for new messages on a schedule, instead of the provider pushing them at you*) reply API. The night agent calls `bot.get_updates()` every ten seconds until it sees your message. No webhook, no server, no template approval.

Webhook vs polling. **Webhook** = provider calls you. Requires you to be reachable at a fixed URL. **Polling** = you call the provider on a schedule. Works from anywhere, even a stateless runner. Polling is slower (you check 10s after a message arrives, not the instant it does), but for our use case 10s is fine.

Section-by-section sending

The full digest is too long for one Telegram message. `send_digest` splits the final document on the section headers (TOPIC FOR TODAY, PRE-KNOWLEDGE, etc.) and sends each block as its own chat message. Reading on a phone feels like nine bite-sized cards, not one wall.

Lesson — delivery is an I/O contract

Every migration only touched delivery/. The pipeline never knew the channel had changed. That's the payoff of keeping I/O out of your business logic.

Automation — GitHub Actions

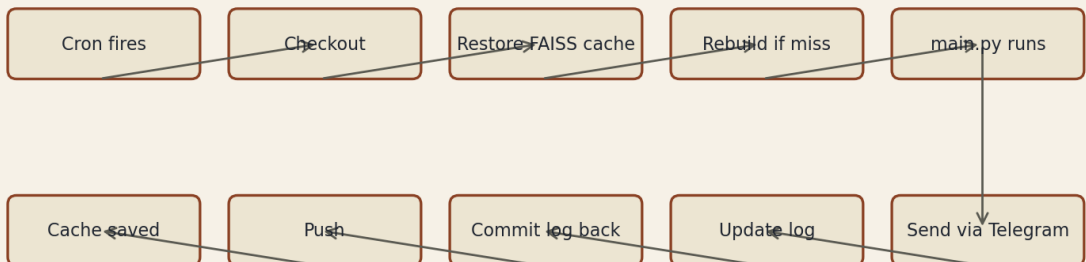
Turn a script you run by hand into a service that runs itself three times a day, for free, with state that survives between runs.

A learning system that requires you to manually run a Python script every morning is not a learning system. **GitHub Actions** (*free CI/CD platform built into every GitHub repo — runs scripts on push, on a schedule, or manually*) runs `main.py` for you on a **cron** (*a way of expressing schedules with five numbers — minute, hour, day-of-month, month, day-of-week*) schedule and gives you a free-tier runner with cache and commit-back.

Runner. *The virtual machine GitHub spins up to execute your workflow. Stateless by default — every run starts fresh. We use the cache and a commit-back trick to give it memory.*

Three schedules

- **02:30 UTC weekdays** → 08:00 IST → daily digest.
- **17:00 UTC weekdays** → 22:30 IST → night agent.
- **03:30 UTC Sunday** → 09:00 IST → weekend distillation.



Stateful loop: cache + log persist between runs

One scheduler run, end to end. Cache + commit-back make consecutive runs see each other.

How a single run flows

Checkout the repo → install with uv → restore the FAISS cache → rebuild it if missing → determine which mode to run by inspecting the cron value → run main.py with secrets injected → after the run, commit weekly_log.json back to main so the next night/weekend run sees it.

Why we commit memory back

GitHub Actions runners are stateless. Without commit-back, every morning's digest would land into a fresh empty log and the night quiz would have nothing to quiz on. Commit-back gives us free durable state with no extra infrastructure.

Warning — keep this repo private

Commit-back puts your topics, debate angles, and quiz scores into git history. If the repo is public, that is public too. main.py prints a one-time reminder at startup unless `DEBATEIQ_SILENCE_PRIVACY=1`.

Caching the FAISS index

Rebuilding the index from PDFs takes a few minutes and burns Gemini embedding quota. The workflow restores the previous run's index from **actions/cache** (a *GitHub Actions feature that saves a directory between runs, keyed by a hash you choose*) keyed by the hash of sources.json, the extractor, and the chunker. If you don't change those, the cache stays warm forever.

Cache key. *The string that identifies a stored cache entry. If the key changes, the cache misses and you rebuild from scratch. We hash the files that affect the index so a config change invalidates the cache automatically.*

Secrets

Anything sensitive lives in the repo's Settings → Secrets → Actions. The workflow names them with the same env-var names the Python code expects. Rotation is a one-place change.

Testing strategy

Five test files, no real API calls. Each stubs the network and the LLM so the suite runs in seconds with zero quota burn.

The codebase has five test files. None of them require network or API keys — they all stub the external surfaces.

Stub / mock / fake. *Loose synonyms for 'a fake version of an external dependency you use in tests'. We use plain Python objects that mimic the real API just enough for the pipeline to run.*

Why no pytest framework

Each file is a script that runs assertions and prints a single success line. CI runs them with `python tests/test_X.py`. We could convert them to pytest but the simpler setup is fine for a project of this size.

What each test covers

- **test_router.py** — every `task_type` routes to the expected model key.
- **test_memory.py** — read/write of `weekly_log.json` round-trips cleanly.
- **test_weekend.py** — `_compute_stats` and `_heuristic_weekend_knowledge` produce the right shapes.
- **test_rag.py** — splitter map, embedding map, retrieval pipeline, source validation.
- **test_daily_e2e.py** — full daily graph with stubbed tools and a fake LLM, asserts every state field is populated.

Stubbing pattern

The e2e test **monkey-patches** (*temporarily overwrites a function or class attribute on an imported module — convenient for testing*) the tool functions

and the LLM pool before importing the graph. The graph then walks the full pipeline using a deterministic fake LLM that recognises each prompt by a key phrase and returns a hardcoded response.

End-to-end (e2e) test. *Exercises the whole system from input to output, not just one function. Slower than unit tests, but catches integration bugs that unit tests miss.*

Tip — fake LLMs are gold

A 50-line fake LLM lets you exercise the entire pipeline in a tenth of a second with zero quota. We add a new case to it whenever a new node lands.

Bug archaeology — what broke and why

Every bug we fixed is a small lesson that pays compounding interest. Read this chapter slowly.

The shape of each entry is the same: **symptom** (what you saw), **root cause** (what was actually wrong), **fix** (the smallest correct change), **lesson** (what you'll keep doing forever).



The four-question template we apply to every bug.

Bug 1: Pre-flight ping doubled LLM calls

Symptom. Every fallback lookup called `invoke('ping')` first.

Root cause. Quota burned twice as fast; some failures still slipped through because the ping passed but the real call 429d.

Fix. Replaced eager check with a lazy proxy that only tries each candidate on real invocation.

Lesson

Defensive checks that cost as much as the thing they protect are anti-defensive.

Bug 2: Topic dict treated as string

Symptom. `_load_topics_config` returned the raw `priority_topics` list of dicts.

Root cause. First node that ran `topic.upper()` crashed with `AttributeError`. Daily mode was unusable without `--topic override`.

Fix. Loader now flattens to `list[str]`; nodes also normalise via `topic_name()` as belt-and-braces.

Lesson

If two parts of the system disagree on a shape, freeze the contract once and enforce it at the boundary.

Bug 3: Article contamination

Symptom. `summarize_node` batched articles into one LLM prompt.

Root cause. `Summary[0]` could quote `article[2]`'s fact. Format node then attributed the wrong title to the wrong content.

Fix. One LLM call per article. Cap content at 2500 chars to keep total tokens under the per-minute limit.

Lesson

Batching is a perf optimisation that costs you correctness. Loop until you have a reason not to.

Bug 4: Reference pages winning rank

Symptom. Heuristic only scored topic-term frequency. Wikipedia hits scored very high.

Root cause. TODAY'S ARTICLE / CASE often opened with an encyclopedia summary instead of a live news piece.

Fix. Added `_is_news_article (+6)` and `_is_reference_article (-8)`; promote news first re-orders after every rank path.

Lesson

Real-world quality lives in metadata, not in token frequency.

Bug 5: Night quiz 413 — too many tokens

Symptom. `_generate_quiz_with_llm` dumped the whole `today_log` into the prompt.

Root cause. 20K+ tokens vs 6-12K TPM caps. Every LLM model rejected; heuristic fallback always ran.

Fix. Trim `today_log` to `topic+key_facts+concepts` before serialising. Skip raw debate angle and english lesson.

Lesson

Free-tier limits are not optional. Audit prompt size before merging any node that touches memory.

Bug 6: FAISS segfaults on Windows

Symptom. `chromadb 1.5.9` native layer crashed under our query load.

Root cause. Whole scheduler run died mid-ingest with exit code 139.

Fix. Migrated to FAISS — leaner persistence, no native crashes on the boxes we run on.

Lesson

Pick the simpler persistence layer unless you need the heavier DB semantics.

Bug 7: Twilio WhatsApp downgraded to SMS

Symptom. Channel prefix on the To: field wasn't propagating in the sandbox.

Root cause. Daily digests arrived as expensive SMS; replies couldn't be polled back.

Fix. Replaced Twilio with Telegram. Cheaper, simpler, pull-based replies.

Lesson

When a provider's failure modes outnumber its features, switch providers.

Bug 8: Broken proxy env vars

Symptom. `HTTP_PROXY=http://127.0.0.1:9` lingered in the shell from an earlier experiment.

Root cause. Every outbound network call from Tavily/Wiki/RSS/DDG silently returned empty.

Fix. Added `clear_broken_local_proxies()` to startup of every network-touching module.

Lesson

Transport errors masquerade as application bugs. Always scrub the environment first.

Bug 9: BM25 corpus empty

Symptom. `build_hybrid_retriever` called `BM25Retriever.from_texts([])`.

Root cause. Every 'hybrid' retriever silently degraded to pure vector. The 40/60 mix was a lie.

Fix. Pull the texts from the FAISS docstore (or `Chroma.get`) and feed BM25 with them.

Lesson

A test that asserts a non-empty retrieval result would have caught this in minutes.

Bug 10: Reply timeout hung the night agent

Symptom. `wait_for_reply` with no env vars looped forever.

Root cause. Local runs without WhatsApp credentials never returned.

Fix. Treat missing credentials as 'no reply' and degrade to bedtime mode.

Lesson

Every blocking call needs a documented exit on a missing precondition.

Bug 11: Gemini model decommissioned

Symptom. gemini-1.5-pro and 1.5-flash were retired without notice in our usage path.

Root cause. Coach lane 404'd on every run.

Fix. Centralised model strings in llm_pool.py; updated to gemini-2.5-pro / 2.5-flash, later moved 'best' to Groq gpt-oss-120b.

Lesson

Provider deprecation is a fact of life. Keep model strings in one place and read the deprecation notices.

Bug 12: LangChain 0.3 dropped pydantic_v1

Symptom. langchain-groq 0.1.x imported from langchain_core.pydantic_v1.

Root cause. Whole project failed to import after bumping langchain-core.

Fix. Bumped langchain-groq, langchain-google-genai, switched EnsembleRetriever import to langchain classic.

Lesson

Pin the whole langchain family together. Upgrading one without the rest is a guaranteed afternoon lost.

Bug 13: Format vs Telegram section drift

Symptom. format_node emits nine sections; delivery/telegram.py knows seven.

Root cause. REBUTTAL DRILLS and WEIGHING LANGUAGE TO USE merge into the previous message instead of standing alone.

Fix. Add the missing labels to `SECTION_TITLES`. Better: derive the title list from `format_node`.

Lesson

Two places that must agree on a list will diverge. Make the list live in one of them.

Bug 14: Naive timestamps in memory

Symptom. `datetime.now().isoformat()` returned local TZ.

Root cause. Dev (IST) and CI (UTC) entries mixed in the same log; weekend windowing was off by hours on Sundays.

Fix. Centralised on `now_iso() = datetime.now(timezone.utc).isoformat()`.

Lesson

Time math without timezones is a quiet bug factory.

Extending the project

Five common changes you'll want to make and the smallest path to each.

Add a new topic

- Open *topics.json*.
- Add an entry under *priority_topics* with *topic*, *debate_frames*, *keywords*, *frameworks*, *live_cases*.
- Optionally upload a related PDF into *knowledge_base/pdfs/topic_pdfs/* and add it to *rag/sources.json*.
- Re-ingest only the new lane: *python rag/ingest.py --only topic_pdf*.

Add a new research tool

- Create *tools/your_tool.py* with a single function *your_tool_search(query) -> list[dict]*.
- Return dicts with the canonical keys: *title*, *url*, *content*, *source*, *published*.
- Import and call it in *agents/research_node.py* inside the *ThreadPoolExecutor*.
- Add a test in *tests/test_tools.py* that asserts your tool returns the right shape on a known query.

Add a new pipeline node

- Decide what it reads and writes. Add the new fields to *core/state.py*.
- Create *agents/my_node.py* with a function *my_node(state) -> state*.
- Always provide a heuristic fallback so the node can never crash the graph.
- Wire it into *graph.py* with *add_node* + *add_edge*.
- Add a fake-LLM case to *tests/test_daily_e2e.py*.

Swap an LLM provider

- Edit only *core/llm_pool.py*.
- Keep the role keys (*fast/balanced/structured/reasoning/long_ctx/best*); change the model strings.
- Add fallback for the new model in *core/fallback.py* if it lives in a different chain.
- Run the smokes — they'll surface any prompt-shape incompatibility immediately.

Change the digest layout

- Edit *agents/format_node.py*.
- If you add or rename a section header, update *delivery/telegram.SECTION_TITLES* in the same commit.
- Run *test_daily_e2e* — it asserts the final doc still contains the expected header.

Debugging playbook

When something breaks, this is the first chapter you open.

Symptoms you'll see in the wild, mapped to the first three places to look. If none of those resolve it, the bug is worth its own archaeology entry.

'I'm not getting the Telegram message'

- Is the GitHub Actions run green? If not, read the last step's logs.
- Are TELEGRAM_BOT_TOKEN and TELEGRAM_CHAT_ID set as repo secrets?
- Did you start a chat with your own bot? Telegram won't deliver to a chat that hasn't opened.

'The digest is empty / fallbacks only'

- Quota: check Groq and Gemini consoles for the day's usage.
- Network: from a local shell run `python -c 'import requests; print(requests.get("https://api.groq.com").status_code)'`.
- Proxy: confirm no HTTP_PROXY env var lurking in the runner.

'RAG returns nothing'

- Does `faiss/` exist on the runner? Cache key may have invalidated.
- Was the index built with the current embedding model? Old embeddings can't be queried with new ones.
- Try `python rag/ingest.py --only knowledge_db` and watch for [Sources] warnings.

'Night quiz LLM call fails every time'

- Print the prompt length. If it's > 12K tokens you're hitting TPM caps.
- Trim *today_log* before serialising — drop *debate_angle* and *english_lesson*.
- Confirm the heuristic fallback is firing; if not, the exception isn't being caught.

'GitHub Actions cron isn't firing'

- GitHub disables scheduled workflows on repos with no recent commits. Push anything (even a comment) to wake it.
- Cron is UTC. Compute the local time from the cron expression before debugging.
- *workflow_dispatch* always works. Use it to verify the workflow itself is healthy.

Glossary

Words that appear throughout this book, defined once and for all.

Agent. A function that takes the shared state, does work, and returns the state. The 'AI' part of 'agentic AI' is just smart functions composed in a graph.

BM25. A keyword-search ranking function from the IR world. Older than embeddings; still wins on exact-term recall. We combine it with vectors in hybrid retrieval.

Callback / commit-back. Pattern where the scheduled job commits its state back to git so the next run can read it. Free durable storage for personal projects.

Chunk. A piece of a source document small enough to embed cleanly. The size and overlap matter more than people realise.

Cron. Unix time expression. `30 2 * * 1-5` means '02:30 UTC, Monday to Friday'. GitHub Actions uses the same syntax.

Embedding. A high-dimensional vector representing the meaning of a piece of text. Similar meanings → similar vectors → close in space.

EnsembleRetriever. LangChain class that runs multiple retrievers and weights their results. We use it to mix BM25 (40%) and vector (60%) on `knowledge_db`.

FAISS. Facebook's vector index library. Fast, persistent, lean. Default backend now that we left Chroma.

Fallback chain. Ordered list of model keys to try when the primary fails. Defined in `core/fallback.py` `FALLBACK_CHAINS`.

Flowable. A ReportLab term for an element that flows down the page — a paragraph, an image, a table. The PDF generator stitches flowables together.

Heuristic. The non-LLM fallback path in every node. Less elegant than the LLM output but always available.

Hybrid retrieval. BM25 + vector retrieval combined with weights. Better than either alone on factual material.

LangGraph. The orchestrator. We declare nodes and edges; it runs them in the right order.

MMR. Maximum Marginal Relevance. A retrieval strategy that penalises redundancy so the chunks you get back cover different angles.

Node. One function in the LangGraph pipeline. In our project each node lives in its own file under *agents/*.

Prompt cache. Disk file storing past LLM responses keyed by (scope, model, prompt) hash. Saves tokens on repeat runs.

RAG. Retrieval-Augmented Generation. Pull your own material into the LLM prompt instead of relying on what the model already knows.

Router. The function that maps `task_type` to a model key. Keeps the choice of model out of every node.

Spaced repetition. Show a topic again only after enough time has passed that you've nearly forgotten it. Our topic selector does a lightweight version.

State. The single mutable dict that every node reads from and writes to. Shape is defined in *core/state.py*.

TypedDict. A Python type that lets you declare a dict's expected keys and value types. The editor checks them.

UTC. Universal Coordinated Time. All timestamps in *memory/weekly_log.json* are in UTC because the runner and your laptop are in different time zones.

uv. A fast Python package manager. Drop-in replacement for pip with better dependency resolution and parallel downloads.

Where to go from here

Read in this order

- `main.py` — the entry point.
- `graph.py` — the wiring.
- `core/state.py` — the contract.
- `topics.json` — the curriculum.
- `agents/research_node.py` — the first real node.
- `rag/retrieval_pipeline.py` — the heart of RAG.
- `agents/coach_node.py` — the heart of the lesson.
- `agents/format_node.py` — how a state dict becomes a digest.
- `delivery/telegram.py` — how a digest becomes a phone notification.
- `memory/weekly_store.py` — how today becomes history.
- `.github/workflows/scheduler.yml` — how all of this happens without you.

If you change one thing this week

Fix the `format/Telegram` section drift (chapter 8, last callout). It's the smallest change with the most visible impact: `REBUTTAL DRILLS` and `WEIGHING LANGUAGE TO USE` will finally arrive as their own Telegram messages instead of gluing onto the section before them.

If you change one thing this month

Trim the `today_log` payload before sending it to the night quiz LLM. That single edit moves the night quiz from 'falls back to heuristic every time' to 'uses the real LLM most nights' — without spending a rupee more.

If you change one thing this quarter

Build a webhook server on Render's free tier so Telegram replies arrive in real time instead of needing the scheduled job to be running. Then the night quiz can be triggered by your reply, not by the clock.

Final note

Every line of code in this project can be read, understood, and rewritten in a single afternoon. The architecture is small on purpose. If something feels too complex to change, the right move is almost always to delete it and rebuild it simpler — not to add an abstraction on top of it.